

University of Westminster
School of Electronics and Computer Science
Fundamentals of Security Technology

Table of Contents

1. Introduction	2
1.1. Importance of File System Analysis in Digital Forensics	2

1.2.	Role of a Forensic Analyst in Investigating Computer Misuse	3
1.3.	Objective of the Report.....	3
2.	Fundamentals of ext4	3
2.1.	Overview of ext4 File System.....	3
2.2.	Advancements and Improvements over ext3.....	3
2.3.	Key Data Structures in ext4.....	3
2.3.1.	Superblock.....	4
2.3.2.	Inode	4
2.3.3.	Block Groups	4
2.3.4.	Data Blocks.....	4
3.	Documentation and Analysis of ext4 Structures	4
3.1.	Comparison of ext4 Concepts with Practical Implementation	4
	Hex View:	7
3.2.	Main ext4 Data Structures and Their Locations.....	10
3.2.1.	Superblock Analysis.....	10
3.2.2.	Inode Structure and Location	11
3.2.3.	Block Group Organization	12
3.2.4.	Data Blocks and File Storage	12
4.	Effect of File Deletion on ext4.....	13
4.1.	Modifications to Data Structures.....	13
4.1.1.	Inode Bitmap.....	13
4.1.2.	Block Bitmap	14
5.	Impact of File Deletion.....	14
5.1.	Persistence of File Data on Disk	15
5.2.	Recovery Potential and Impact on Digital Forensics.....	15
6.	Challenges in ext4 Forensics	15
6.1.	File Fragmentation and Data Reconstruction	15
6.2.	Incomplete Metadata and Hidden Data	15
6.3.	Recovery of Deleted Files and Artifacts	15
7.	Reflective Conclusion	15
7.1.	Summary of Findings.....	15
7.2.	Importance of ext4 File System Analysis in Digital Forensics	15
7.3.	Future Developments in ext4 Forensics	16
	References	16

1. Introduction

1.1.Importance of File System Analysis in Digital Forensics

Digital forensics investigations rely heavily on file system analysis. With the proliferation of cybercrime and computer abuse, forensic analysts are increasingly being called upon to unearth data from computers and other electronic devices. The ability to retrieve useful information and recreate what went on within a compromised

system depends on one's familiarity with the structure and functioning of file systems.

Forensic analysts can determine whether or not vulnerabilities in digital storage systems have been exploited by examining file systems. This evaluation aids in comprehending the depth of the breach, recognizing the breadth of the unauthorized access, and estimating the possible loss or harm brought on by the abuse of a computer. Legal admissibility of digital evidence may be ensured by file system analysis, which also helps establish a chain of custody [1].

1.2.Role of a Forensic Analyst in Investigating Computer Misuse

A forensic analyst's expertise is crucial in the investigation of hacking and other forms of computer crime. To ascertain the extent of the occurrence, they must gather, store, and examine digital data. Fundamental to their job is the examination of file systems, which reveals information about how a computer stores and retrieves data. File system forensics involves the analysis of data structures, metadata, and file allocation information using specialized tools and methods. They can detect tampering, unauthorized access, or efforts to hide evidence by learning the file storage and retrieval processes. Moreover, file system analysis enables the restoration of deleted or concealed files, which may aid in the discovery of information that may be vital to the inquiry.

1.3.Objective of the Report

This research aims to examine the examination of the ext4 file system from both theoretical and practical perspectives. The purpose of this paper is to educate readers on the inner workings of the ext4 file system and the difficulties forensic analysts have when trying to recover data from a hard drive that uses the ext4 file system after files have been erased.

We will learn about the internals of a hard drive and the ext4 file system by actually using one. We'll examine the fundamental ext4 data structures, discover where files are stored inside the file system, and learn about the alterations made when files are created, moved, or destroyed. We will also cover the difficulties that forensic analysts have when analyzing an ext4 file system and analyze the effects that deleting a file has on its contents. By the conclusion of this study, readers will have a better understanding of the problems forensic analysts may have when attempting to recover deleted files from an ext4 file system, as well as the repercussions of deleting files. The purpose of this paper is to add to the body of information on analyzing file systems for digital forensics purposes.

2. Fundamentals of ext4

2.1.Overview of ext4 File System

Many Linux distributions make use of the ext4 file system, often known as the fourth extended file system. It's an improvement over its predecessor, ext3, with better speed, scalability, and reliability. With Ext4, you may have bigger files, a larger file system, and quicker check and repair times for your files.

Ext4 uses a journaling file system strategy to protect data and metadata by recording changes locally before writing them to disc. In the event of a system crash or power outage, the file system may be recovered using this.

2.2.Advancements and Improvements over ext3

Ext4 is a more reliable and efficient file system than its predecessor, ext3, thanks to a number of new features and enhancements. Important new features include:

- a. Increased file system size: Ext4 is compatible with file systems that are much bigger in size, enabling partitions and file systems to be created with a capacity of up to 1 exabyte (1 million terabytes).
- b. Extents: Ext4 allows for files up to 16 terabytes in size, facilitating the effective storing and handling of massive data sets.
- c. Multiblock allocation: Ext4 uses multiblock allocation, which allots numerous blocks at once to improve speed and decrease disc fragmentation.
- d. Delayed allocation: When a file is created or expanded using Ext4, data blocks are not instantly allocated. This technique is known as delayed allocation. Instead, disc space allocation is postponed until it's really needed.

2.3.Key Data Structures in ext4

Ext4 uses a number of different data structures to handle and store data. File system analysis requires an appreciation of these data structures.

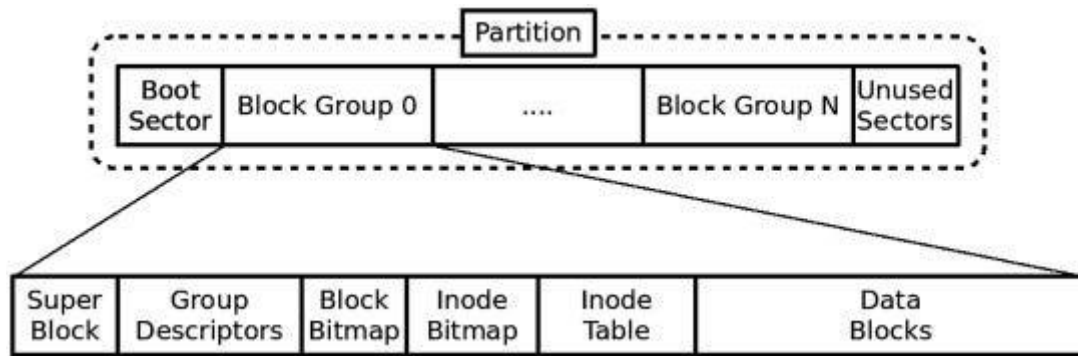


Figure 1: Key data structures of Partition

The key data structures in ext4 include:

2.3.1. Superblock

The superblock is an important data structure that must be created at the start of every file system. The block size, number of inodes, number of block groups, and total file system size are all included in this information. The superblock is the primary entry point to all other file system data structures.

2.3.2. Inode

A file or directory is represented in the file system by an inode (index node), a data structure. File metadata includes information such as the file's access control settings, owner information, size, modification and creation dates, and pointers to the actual data blocks that constitute the file's content. Inodes are stored in inode tables that are part of larger block groups.

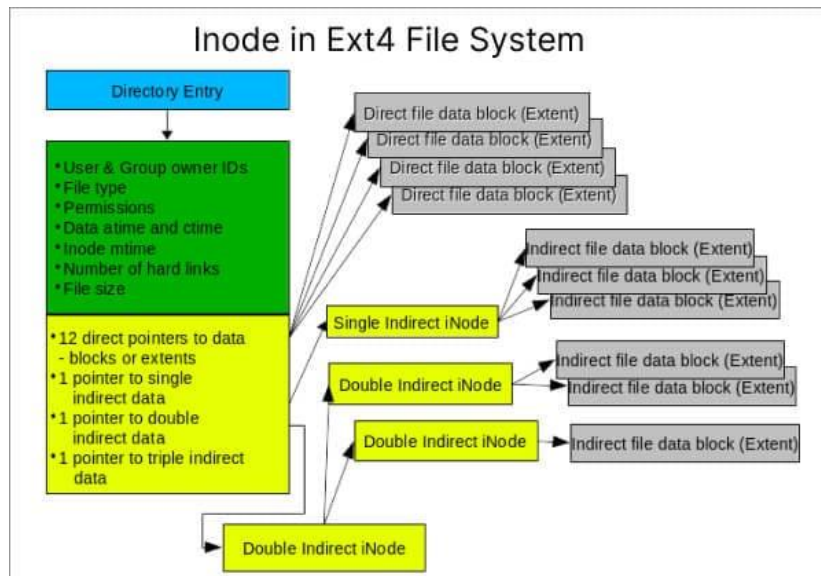


Figure 2: Inode in ext4 File System

2.3.3. Block Groups

The file system's block groups are the logical containers for organizing data blocks and inodes. Data blocks and its associated inodes are organized into groups called block groups. The data and metadata of a file system may be distributed over the disc using block groups, which boosts efficiency and enables simultaneous access to numerous files.

2.3.4. Data Blocks

The information contained in a file is represented by data blocks. Ext4 allocates the disc space to hold the files in blocks of predetermined sizes, usually 4KB or 8KB. The data blocks are linked together to form a file's content, and the inode contains pointers to these data blocks.

Understanding these key data structures is essential for analyzing the ext4 file system and tracing the location of files within the system. The interplay between the superblock, inodes, block groups, and data blocks facilitates efficient file storage, retrieval, and management in ext4.

3. Documentation and Analysis of ext4 Structures

3.1. Comparison of ext4 Concepts with Practical Implementation

In the first step, I have created the partition ext4 with size of 987 (last 3 digits of ID). And then formatted it with the 16k unit size/sector size.

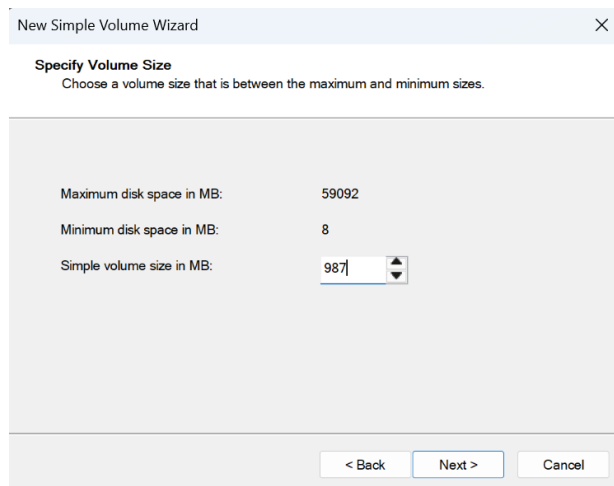


Figure 4: New Simple Volume Size

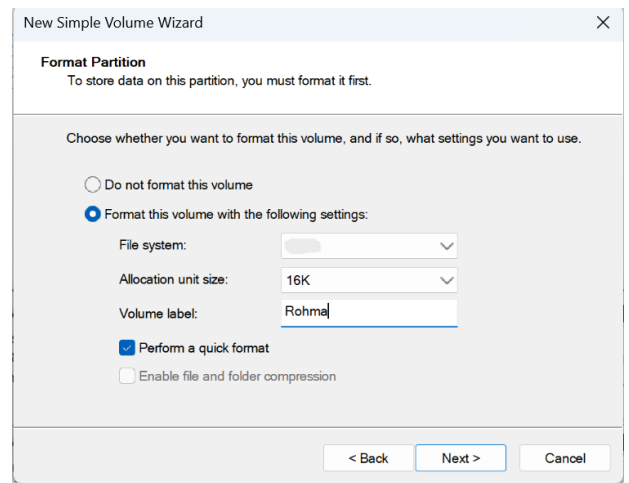


Figure 3: Format Partition

Assigned the partition a disk letter D with name of (First name). And remaining space with letter E named (Last name).

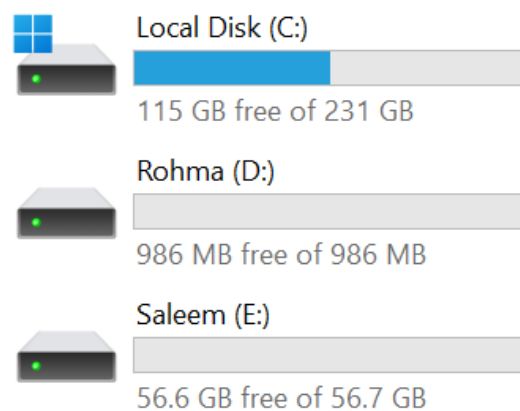


Figure 5: Partiton Disk created

Now I have used EaseUS to check the partition details and convert the partition to ext4 as windows 11 don't give manually option to make ext4 partitioning.

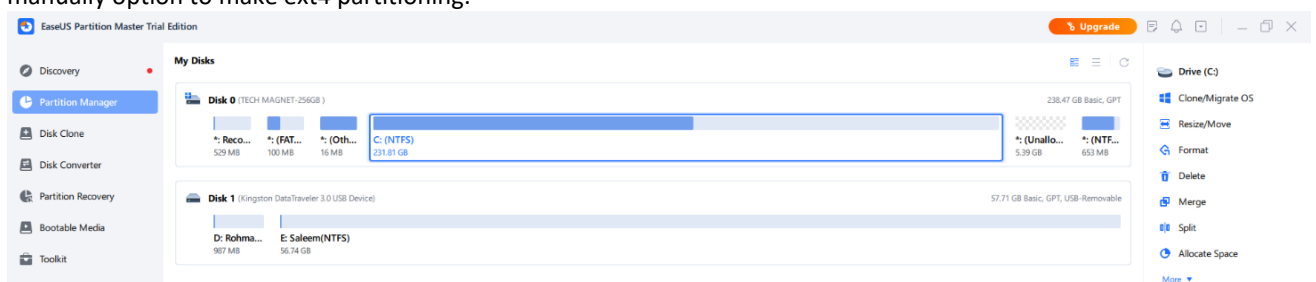


Figure 6: EaseUS Partiton

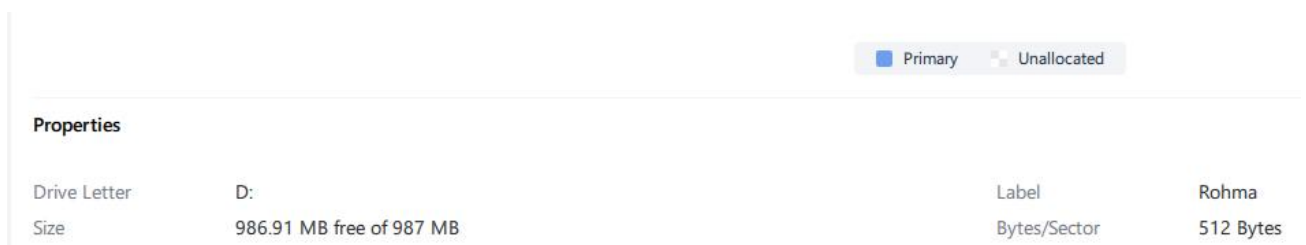


Figure 7: Properties of the Partition

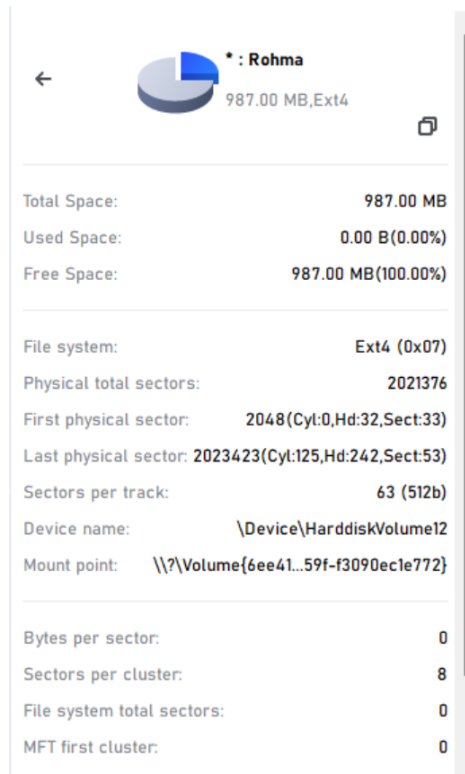


Figure 8: Disk partition ext4 Properties

Here are the details of the disk that I have partitioned. The above screenshot provides details of free space, file system, physical total sectors, sectors per track, etc.

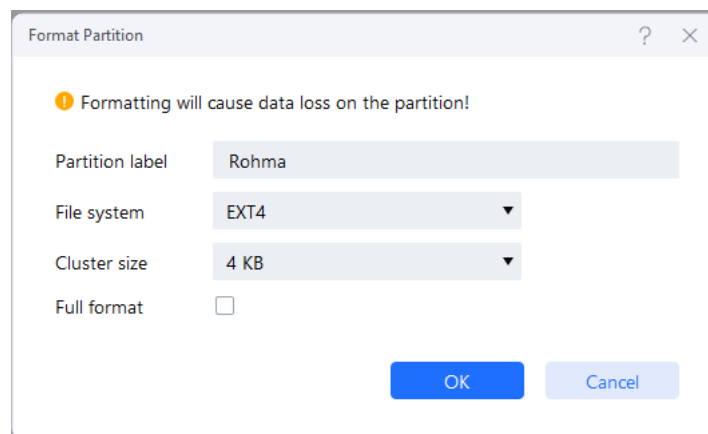


Figure 9: format Partition through EaseUs

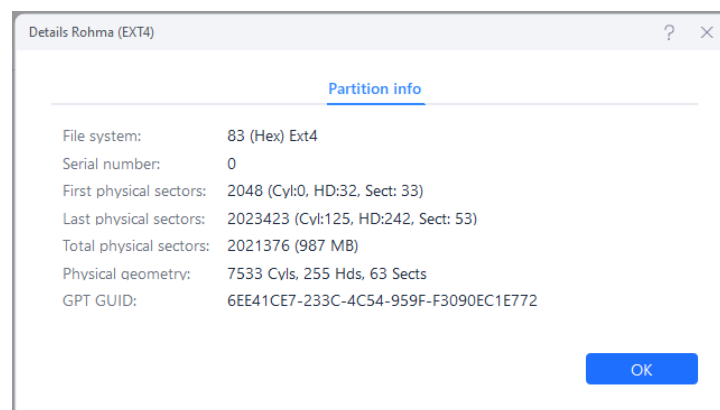


Figure 10: Details of the Disk

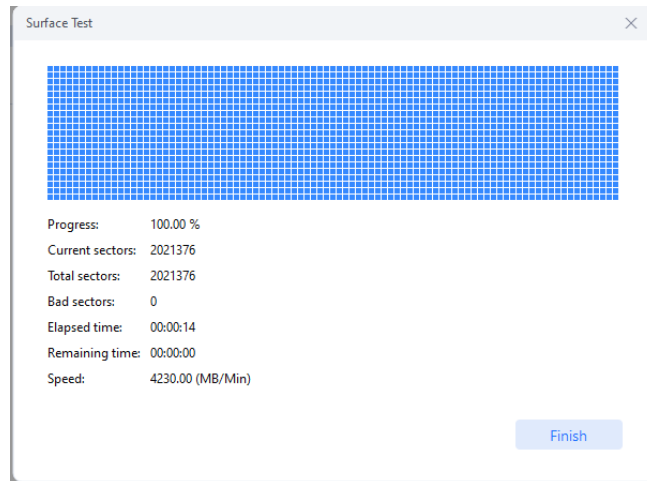


Figure 11: Surface Test of the Disk

Here are the data of the complete GPT formatted disk. Each partition entry in the GPT format contains various attributes, including the partition type GUID, unique partition GUID, starting and ending LBA (Logical Block Address), and partition name. The partition type GUID identifies the purpose or type of the partition, such as an EFI system partition or a data partition. The disk complete interpreter details are here in below details;

Data Interpreter (Offset: 240)	
Name	Value
✓ GPT	
Signature (EFI PART)	]
Revision No	0x5EB7DFC
Header Size (should be 92)	3 950 902 176
Header CRC32	2 340 271 104 (= 2285421 kb)
Reserved	1 083 747 568
Primary LBA (should be 1)	-4 967 835 364 769 657 740
Backup LBA	- 170 872 992 961 855 481
First Usable LBA	7 357 136 964 113 853 309
Last Usable LBA	5 982 557 554 032 141 920
Disk GUID	{00106866-0001-42B4-B280-8A166...
Partition Entry LBA (should be...)	6 369 875 919 555 138 509
MaxNo of Partition Entries	1 634 097 254
Size of Partition Entry	1 577 300 338
Partition Entry Array CRC32	1 228 957 400
✓ Partition Entry (sector size: 5...)	
✓ Entry #1	
Partition Type GUID	{185D8B66-0366-66C3-5B66-D3E86...
Unique Partition GUID	{3C03458A-7507-C641-46EF-03EB3...
Starting LBA	6 297 812 827 824 862 837
Ending LBA	399 413 025 154 713 475
Attribute Bits	200 272 768 254 598 949
Partition Name	吹曳晨晚牌茂~□的些紋F髹彦兹,触子...
✓ Entry #2	
Partition Type GUID	{0F5A665B-C182-83FE-C404-E97FF...
Unique Partition GUID	{00000000-0000-0000-0000-000000...
Starting LBA	0
Ending LBA	0
Attribute Bits	0
Partition Name	
✓ Entry #3	
Partition Type GUID	{00000000-0000-0000-0000-000000...
Unique Partition GUID	{41B05066-48E8-66FD-5883-C4048...
Starting LBA	1 216 184 546 551 918 718

Figure 12: Data Interpreter view of Disk

Hex View:

Hexadecimal analysis of the GPT-formatted ext4 disk with a size of 987 GB reveals valuable insights into the structure and organization of the file system. Looking at the disc image in binary allowed the hexadecimal analysis to pinpoint certain patterns and data structures.

Preview	
Encoding	ANSI (Windows)
Mode	Hex
Data Interpreter	Go To Search
0097D040:	CD F4 01 4E 92 A3 E5 1C 1C 6F 0A 36 63 04 36 9D Íô.N'£ă..o.6c.6
0097D050:	CA AB BF 29 00 11 86 CE 50 E5 DA 36 8E 88 51 FE Ê«¿)...)ÎPăÚ6Ž^Qb
0097D060:	72 FB 10 DF 98 56 54 94 7B FD 06 3F AE 52 D7 57 rû.ßVT" {ý.?@R×W
0097D070:	08 60 98 A2 0A D5 2E 08 13 C4 F2 6F 79 5B 20 5B .`¢.Ö...Ăðoy[[
0097D080:	C7 EE B2 E9 10 F9 44 2E F5 BE A9 B5 BC 1E BB B1 Çî²é.ùD.ö³µ¹.»±
0097D090:	1C 81 90 C5 D1 26 8D 92 38 8E FE 74 F5 78 22 8B .ĂÑ&'8Žptðx"<
0097D0A0:	BB AF FA 03 6A DC 98 2E 3F E9 98 75 BD 5A C0 DE »-ú.jÛ:~?éü½ZÀP
0097D0B0:	09 0B 83 AE 86 E5 FB 56 12 D1 C0 42 85 78 9A DF ..f@+âúV.ÑÀB...xšß
0097D0C0:	9C CB 4F 8E 3A 27 A7 44 19 F7 78 E6 E8 AD 85 97 œËÖŽ: 'SD.÷xæ-...-
0097D0D0:	0D 7D FE C3 26 4D 7C 46 11 55 FF 33 E6 DC 34 39 .}pĂ&M F.Uÿ3æÜ49
0097D0E0:	51 BA 2C 35 0C 3B CE 19 56 8B 05 3F 65 47 DF A2 Q°,5.;î.V< .?eGß¢
0097D0F0:	AF B2 C5 60 86 19 3C 75 89 E0 DB D3 E6 8F 87 25 -²Ă+<.u%âÜÓæ±%
0097D100:	44 C6 47 70 C0 59 5C 10 35 47 EF A0 6C AC 3F 06 DÆGpÀÿ\ .5Gi 1-?.
0097D110:	05 4A B8 0F 86 2A FE DF C0 F9 C2 7F 23 D8 F2 D0 .J,.+*pßÀùĂl#øð
0097D120:	AB 6F 98 2B 43 36 99 5D 53 F7 30 F5 82 3F 24 E2 «o†C6™]S÷øð,?šâ
0097D130:	5B E1 1E E0 8A 2A F1 D8 B2 C4 2A 0B EE 7E 84 34 [á.àš*ñø²Ă*.i~,,4
0097D140:	58 36 5E C4 98 89 52 91 79 34 A9 D1 35 D8 E6 31 X6^ĂžR'y4©Ñ5øæ1
0097D150:	17 33 7A 5F B1 61 34 A3 28 48 D3 13 EC AB 5B 94 .3z_±a4£(HÓ.ì«["
0097D160:	F9 88 75 5C 4B 03 8F 88 E9 89 DB 64 19 46 58 14 ù^u\K.^é%Ūd.FX.
0097D170:	6F 65 FC E8 09 57 48 07 B1 22 4C 2D DA 0D A4 76 oeüè.WH.±"L-Ū.µv
0097D180:	94 E8 48 E6 0C B3 7A 19 AC 7F 27 6B 3A 94 83 D2 "èHæ.³z.-ll'k:"fÒ
0097D190:	17 A6 71 C0 9F 7A 97 B0 66 AB C6 73 34 23 AF FD .!qĂŸz-°f«Æs4#-ý
0097D1A0:	F1 A5 27 41 4F 6C EA D6 09 8B 62 8E 17 FF 62 82 ñ¥'AOlêÖ.<bŽ.ÿb,
0097D1B0:	B1 5A 3F 77 08 C0 DA 07 8A E8 E9 40 BA 66 A0 C1 ±Z?w.ÀÛ.Šèé@°f Ă
0097D1C0:	15 7F 27 B6 1D CF C6 D5 A3 60 8F EA 44 37 4B 2D .ll'q.İÆÖ£`èD7K-
0097D1D0:	05 27 30 50 BF 0D F5 FE CD BB DC 6C 7B E2 6B 32 .'0P¿.öþÍ»Ūl{ák2
0097D1E0:	58 81 47 A7 3E 1E E2 4E AC AA D8 E2 19 2E F4 F5 XG\$>.âN-ªøâ..ôö
0097D1F0:	81 6E D7 92 DD CA 1A 0F C9 9E 81 00 06 E0 FC 83 n×'ÝÊ...Éž...àüf
0097D200:	92 9F 5C 49 B2 77 60 52 DF 08 B2 94 EC 14 A6 5B 'Ý\I²w`Rß.²"ì.¡[
0097D210:	F3 7A 36 3C F8 A2 C1 C5 59 43 E8 6D A4 E1 90 83 óz6<ø¢ĂÂYCèm¼áf
0097D220:	BA E4 54 6C 14 CB 06 D0 25 78 20 E7 F5 F6 F3 10 °âTl.Ě.Đ%x çöóó.
0097D230:	C5 41 1C 2D DD EE 3E 7E AE EF AF B0 A1 99 78 37 ÅA.-Ýi>~©İ°;™x7
0097D240:	AC 5B 1A 69 B1 9B C8 C6 9F 2C 78 EE 4F 9D 54 52 ¬[.i±>ÈÆŸ,xíOTR
0097D250:	E3 D9 3A 6E 12 BB 39 9B CE F3 74 72 D4 23 04 09 äŪ:n.»9>İótrÔ#..

Figure 14:Hexadecimal bits

The precise ext4 partition and its location might be established by inspecting the GPT partition entries. This enabled for the study of the ext4 file system inside the allotted partition and gave critical data for further research. The ext4 superblock, which holds essential file system information, was subsequently discovered during the investigation. The file system's block size, inode count, and the number of block groups are all pieces of data that may be found in the superblock. These specifics are useful for comprehending the ext4 file system's general properties and laying the groundwork for future investigation.

Preview														
Encoding	ANSI (Windows)				Mode	Hex				Data Interpreter	Go To	Search		
016803F0:	96	28	FD	A4	BF	D5	95	9B	F2	05	E0	BD	3A 57 C7 EE	– (ýµçÖ•>ò.à½:WÇî
01680400:	0C	8B	18	C1	40	E3	B5	A5	73	F8	FC	4D	3F EB FE A2	.<.Ã@ãµ¥søüM?ëþç
01680410:	FC	06	4A	AD	28	B3	DC	AD	80	B8	33	8A	E7 FE 72 3B	ü.J– (³Û–ē,3Šçþr;
01680420:	5A	74	65	74	0D	2F	92	4E	C0	25	B1	F0	E9 D9 00 E1	Ztet./'NÀ%±ðéÛ.á
01680430:	21	B0	D5	60	44	98	63	B7	24	49	00	22	E8 9B F4 29	!°Ö`Dç·\$Î."è>ð)
01680440:	E3	76	2D	F8	3D	A9	A7	73	13	1E	1B	96	32 62 39 8C	äv-ø=©\$s...–2b9œ
01680450:	F4	9A	53	8C	26	66	1D	AE	1D	80	9D	2F	5C 1B 14 B5	ôšSœ&f.®,€/\\..µ
01680460:	29	8D	55	60	F5	72	75	1E	4C	FB	68	D7	FD 39 25 40	)U`õru.Lûh×ý9%ê
01680470:	5A	6E	7E	86	8D	21	FF	2F	AB	6B	FF	1A	0E B2 61 57	Zn~†!ÿ/«kÿ..²aW
01680480:	0F	C2	05	9C	09	FE	07	FE	09	2B	06	58	47 A2 46 E3	.Ã.œ.p.p.+·XGçFă
01680490:	E7	4A	A8	31	3D	40	8A	2F	09	B4	37	73	8E 50 38 D6	çJ`l=êŠ/.´7sŽP8Ö
016804A0:	C2	4F	F8	E2	9C	6C	8B	83	F8	E3	12	4A	46 9F DA 31	ÃOøâœl<føă.JFÝÚl
016804B0:	41	0B	45	3B	FB	0F	01	B7	A7	C4	46	55	B0 E8 F4 45	A.E;û...·ŠÄFU°èøE
016804C0:	41	24	2C	96	D1	A3	AA	BD	F8	08	C5	26	48 FA C8 C3	A\$,–Ñ£ª½ø.Ã&HúÈĂ
016804D0:	B1	83	C7	3E	73	98	BC	AA	24	1E	7C	FE	80 7A 37 87	±fç>s¼ª\$. þez7†
016804E0:	F8	87	48	89	81	D9	AE	29	01	74	DB	F8	6C D5 DD 62	ø†H½Û®).tÛølÖÝb
016804F0:	0F	A6	AA	0E	93	34	F0	A9	0E	82	A7	DB	63 78 F4 B9	. ª."4ð®,ŠÛcxð¹
01680500:	68	70	5A	9E	7B	A4	C7	3C	AF	38	08	9D	B3 52 13 25	hpZž{µç<⁻8.³R.%
01680510:	EC	62	74	F7	F8	40	E4	5E	84	F7	D8	FB	34 A1 B3 CD	ìbt÷øèä`„÷øû4;³í
01680520:	09	F3	78	89	68	54	13	72	91	25	5F	1E	23 B6 FB C9	.óx%hT.r`%#.¶ŮÉ
01680530:	2A	CC	00	7D	19	87	63	5D	70	BD	3E	79	F7 D6 A3 AC	*ì.).+ç]p½>y÷Ö£¬
01680540:	10	71	A1	E2	01	F5	C3	89	4D	DA	E0	0B	2B 41 47 2C	.q;â.õĂ%MÚà.+AG,
01680550:	46	B7	7E	0A	7A	75	3B	71	E3	6C	A4	73	FF 19 52 3C	F~.zu;qălmsÿ.R<
01680560:	81	AB	39	CF	80	7E	D2	31	60	70	7A	E8	F8 AE EC	<9İē~0l`pzèèø@ì
01680570:	51	1C	45	6A	51	DB	1B	67	07	38	AF	F3	5A 80 D1 5E	Q.EjQÛ.g.8⁻óZEñ^
01680580:	15	DE	81	7E	15	0F	05	82	32	D6	7C	7E	25 FE 55 3A	.B~...,20 ~%pU:
01680590:	ED	60	C4	40	D1	BE	31	EE	72	06	03	3C	52 B5 6C 11	í`À@Ñ½lîr...<Rµl.
016805A0:	54	4F	88	81	02	B0	BF	81	B8	10	B1	70	08 FB 03 AC	TO^°.°¿,.±p.û.¬
016805B0:	38	93	A0	8B	A6	51	2A	93	8F	AB	ED	C6	3C 4D A8 4A	8" < Q*"«íÆ<M`J
016805C0:	B0	A6	6F	9C	60	62	4C	19	DA	12	B8	10	24 69 F5 81	° œe`bL.Ú.,.Şið
016805D0:	ED	78	9B	CC	58	CD	DA	A1	12	52	88	4A	15 68 BB AA	íx>ìXÍÚ;R^J.h»ª
016805E0:	C9	62	91	44	3A	47	5D	06	9A	C2	BA	CB	03 2E 42 3E	Éb`D:G].šÂ°Ė..B>
016805F0:	E6	A4	5E	1A	D3	78	D5	57	05	C9	99	1D	C2 00 D0 D3	æµ^·ÓxÖW.É™.Ã.ĐÓ
01680600:	FC	82	6E	31	ED	38	FC	CC	96	8D	8C	B5	9E 5A DB 2E	ü.nlî8üİ–œužZÛ.

Figure 15: Hexadecimal view of the disk

3.2.Main ext4 Data Structures and Their Locations

3.2.1. Superblock Analysis

By analysing the superblock, we could see how the file system's block groups were structured. By logically organising data blocks and their associated inodes into block groups, files may be stored and retrieved quickly and easily. Knowing how the ext4 file system is organised into block groups is crucial for exploring and reading files inside the system. The superblock analysis is shown in the below snapshot from the data interpreter:

Data Interpreter (Offset: 0)		
☐ Name	☐ Value	
▼ Ext4_super_block		
Inode count	1 167 095 531	
Block count (lo)	1 413 563 992	
↳ Block count	-4 967 835 013 119 654 312	
Reserved block count (lo)	2 105 376	
Free block count (lo)	0	
Free Inode count	0	
First data block	0	
Block size (0=1K, 1=2K, 2=4K)	0	
Fragment size (same)	0	
Blocks per group	0	
Fragments per group	0	
Inodes per group	0	
Last mount time	01.01.1970	00.00.00
Last write time	01.01.1970	00.00.00
Mount count	0	
Maximal mount count	0	
Magic signature (53 EF)	0	
File system state	0	
Behavior when detecting errors	0	
Minor revision level	0	
Time of last check	01.01.1970	00.34.08
Max. time between checks (s...)	0	
OS (0: Linux)	2 021 376 (= 1974 kb)	
Revision level	0	
User ID for reserved blocks	128	
Group ID for reserved blocks	0	
▼ Extended Superblock Section		
First non-reserved Inode	512	
Inode size	640	
This superblock's block gr...	0	
Flags	0xF6AC	
↳ Has journal	1	
Incompatibility Feature Fla...	0x4	

3.2.2. Inode Structure and Location

To create an ext4 file system, type "mkfs.ext4" into a command prompt. The command "mkfs" means "make file system," and the extension "ext4" indicates the desired file system type.

The "/dev/sdb" device will be used to install the ext4 file system. Linux uses device files in the "/dev" directory to represent storage devices. Here, "/dev/sdb" refers to a particular mass storage device. This may be a hard drive or a USB flash drive, for example.

Create an ext4 file system on the given device (/dev/sdb) in Linux using the "mkfs.ext4 /dev/sdb" command.

```
root@main:~# mkfs.ext4 /dev/sdb
mke2fs 1.45.7
Discarding device blocks: done
Creating filesystem with 16777216 4k blocks and 4194304 inodes
Filesystem UUID: e8283c00-4817-4f96-ac3d-2c645f37e379
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
    4096000, 7962624, 11239424

Allocating group tables: done
Writing inode tables: done
Creating journal (131072 blocks): done
Writing superblocks and filesystem accounting information: done
```

Figure 16: Creating filesystem with 16777216 4k blocks and 4194304 inodes

- For a file system with a block size of 4KB, the value "16777216 4k blocks" specifies a total of 16,777,216 blocks. The term "block size" describes the smallest data storage "block" in a certain file system.
- The value "4194304 inodes" indicates the desired maximum number of new inodes for the file system. Inodes are data structures that keep track of information about files, such as who owns them, when they were last modified, and where the data blocks that make up the file are located.

The size and structure of the newly constructed ext4 file system may be deduced from these specifications. Files may be stored and managed on the provided device ("/dev/sdb") according to the file system's capacity and organisation, which is determined by the number of blocks and inodes.

3.2.3. Block Group Organization

```
root@main:/disk# touch a.txt
root@main:/disk# stat a.txt
  File: a.txt
  Size: 0          Blocks: 0          IO Block: 4096   regular empty file
Device: 810h/2064d Inode: 12         Links: 1
Access: (0644/-rw-r--r--)  Uid: (  0/   root)   Gid: (  0/   root)
```

Figure 17: Created the file with zero byte named as a.txt

File: a.txt: This line lists the file's name, "a.txt."

Size: 0: This line states that the file is 0 bytes in size. The file is empty since you just created it with the touch command.

Device: 810h/2064d: This line displays the file's inode number and device number. The device number is shown in decimal (2064d) and hexadecimal (810h).

Blocks: 0: This line shows how many blocks have been assigned to the file. Since the file is empty in this instance, it displays 0 blocks.

Inode: 12: This line shows the file's allocated inode number. The inode is a data structure that houses the file's meta data.

(0644/-rw-r--r--) Access: The file permissions are shown in this line. The octal notation for the authorization mode is 0644. The owner (root) has read and write access to the file, but the group and other users only have read access.

Uid: (0/root): The user ID (UID) and username of the file's owner are shown on this line. The UID in this instance is 0, which normally represents the root user.

Gid: (0/root): This line displays the file's group name and group ID (GID). The root group is normally represented by the GID of 0, which is.

IO Block: 4096: The block size utilized for file I/O operations is shown on this line. It is set to 4096 bytes in this instance.

Links: 1: The number of hard links to the file is displayed on this line. It just has one link because it is a recently generated file.

normal empty file: The file type, which in this case is a regular empty file, is indicated by the line.

```
root@main:/disk# echo -n "a1b2c3" > a.txt
root@main:/disk# stat a.txt
  File: a.txt
  Size: 6          Blocks: 8          IO Block: 4096   regular file
Device: 810h/2064d Inode: 12         Links: 1
```

Figure 18: After adding six characters into the disk file a.txt

3.2.4. Data Blocks and File Storage

```
root@main:/disk# filefrag -v a.txt
Filesystem type is: ef53
File size of a.txt is 6 (1 block of 4096 bytes)
ext:      logical_offset:      physical_offset: length:   expected: flags:
  0:         0..          0:      34306..    34306:      1:      last,EOF
a.txt: 1 extent found
```

Figure 19: Data Blocks and file storage

- The "filefrag" command is used to examine file fragmentation on a computer's hard drive. The number and position of fragmented extents, as well as other information about the file's fragmentation condition, are provided.
- The "verbose" (or "-v") option shows output for every section of the file in great detail. Extent number, beginning and ending block numbers, and disc location are just some of the further details that may be gleaned from the "filefrag" command.
- A text file with the name "a.txt" will serve as the subject of our fragmentation analysis. The name of the text file in question is "a.txt".

In depth fragmentation data for the given file "a.txt" will be printed when the "filefrag -v a.txt" command is run. The file will be broken down into extents, which are logical units of storage, for easier analysis. The output will include the extent number, start and end block numbers, the number of blocks within each extent, and the

physical disk location.

The "filefrag" command is used to display the fragmentation information for a file. It provides details about the allocation of data blocks on the file system for a specific file.

4096 bytes / 512 bytes = 8 blocks.

4. Effect of File Deletion on ext4

4.1. Modifications to Data Structures

4.1.1. Inode Bitmap

In a simple term, inode is the identity card of the file. In Linux, every file must have an inode. Because inodes map the blocks to physical sectors. I would like to remind you that the data in the file is actually stored in blocks.

One inode can be used by the multiple files (hard-link)

Inode is actually a data structure that holds some information about the file. e.g.: access, change and creation time, access rights (chmod), uid, gid, number of blocks allocated to the file, number of hard links (not symbolic links, every symbolic has another inode), etc.

An inode doesn't hold the file name. Because one inode can be used by multiple files due to hard-link. Therefore, if you have created a hard link, the inode can't be released even if the first file is deleted. Because the inode is still in use. The command "ls -li" is used to list files and their inode numbers in a directory. Each file in the directory is displayed along with its respective inode number.

```
root@main:/disk# ls -li
total 4
11 -rw-r--r-- 1 root root [redacted] 22:16 a.txt
root@main:/disk# ln a.txt b.txt
root@main:/disk# ls -li
total 8
11 -rw-r--r-- 2 root root [redacted] 22:16 a.txt
11 -rw-r--r-- 2 root root [redacted] 22:16 b.txt
root@main:/disk# ln a.txt c.txt
root@main:/disk# ls -li
total 12
11 -rw-r--r-- 3 root root [redacted] 22:16 a.txt
11 -rw-r--r-- 3 root root [redacted] 22:16 b.txt
11 -rw-r--r-- 3 root root [redacted] 22:16 c.txt
root@main:/disk# rm a.txt
root@main:/disk# ls -li
total 8
11 -rw-r--r-- 2 root root [redacted] 22:16 b.txt
11 -rw-r--r-- 2 root root [redacted] 22:16 c.txt
root@main:/disk# rm b.txt
root@main:/disk# ls -li
total 4
11 -rw-r--r-- 1 root root [redacted] 22:16 c.txt
root@main:/disk# rm c.txt
root@main:/disk# ls -li
total 0
```

Figure 20: The first column is the inode number, the third column is the number of hard links

```

root@main:/disk# ls -li
total 16
12 -rw-r--r-- 1 root root      22:44 a.txt
13 -rw-r--r-- 1 root root      22:44 b.txt
14 -rw-r--r-- 1 root root      22:44 c.txt
11 drwx----- 2 root root 16384 22:44 lost+found
root@main:/disk#
root@main:/disk# rm b.txt
root@main:/disk# touch d.txt
root@main:/disk# ls -li
total 16
12 -rw-r--r-- 1 root root      22:44 a.txt
14 -rw-r--r-- 1 root root      22:44 c.txt
13 -rw-r--r-- 1 root root      22:45 d.txt
11 drwx----- 2 root root 16384 22:44 lost+found

```

Figure 21: The first column is the inode numbers

When you create a new directory, its inode will be higher. When you create a new file in the directory, the inode of the file will be higher than the inode of the directory. If you move a file to the directory, the inode of the file is kept:

```

root@main:/disk# tree --inodes
.
├── [ 12]  a.txt
├── [1572865] example
│   └── [1572866] b.txt
└── [ 11]  lost+found

2 directories, 2 files
root@main:/disk#
root@main:/disk# mv a.txt example/
root@main:/disk#
root@main:/disk# tree --inodes
.
├── [1572865] example
│   ├── [ 12]  a.txt
│   └── [1572866] b.txt
└── [ 11]  lost+found

```

Figure 22: tree view with inodes

The tree command is a useful tool for visualizing directory structures. When used with the --inodes option, it displays the inode numbers associated with each file and directory.

4.1.2. Block Bitmap

When you create a new filesystem, your first file will be allocated in inode 12. Because inode 11 will be allocated to the lost+found directory. If you delete this directory and create a file, the new file will have inode 11.

Linux kernel tends to allocate lower inodes for files. If you delete a file and create another file, the inode of the deleted file is allocated to the new file:

```

Group 509: (Blocks 16678912-16711679) csum 0x2708 [INODE_UNINIT, BLOCK_UNINIT]
Block bitmap at 16252941 (bg #496 + 13), csum 0x00000000
Inode bitmap at 16252957 (bg #496 + 29), csum 0x00000000
Inode table at 16259616-16260127 (bg #496 + 6688)

```

Figure 23: Part of dumpe2fs /dev/sdb output

Same for the block bitmap. Its value is 1 if a block is used, 0 otherwise.

5. Impact of File Deletion

When a file is deleted in the ext4 file system, certain changes occur within the file system that affect the availability and visibility of the deleted file. Understanding the impact of file deletion is crucial for forensic analysts during investigations [3].

5.1.Persistence of File Data on Disk

Even though a file is deleted in the ext4 file system, the actual data associated with the file remains intact on the disk until it is overwritten by new data. The file system marks the blocks previously occupied by the deleted file as available for reuse but does not immediately remove the file's content from the disk.

Data from deleted files may be recovered if they remain accessible on the disc until they are rewritten. This has consequences for digital forensics investigations, since this means that deleted information may be recovered and useful evidence extracted using specialized tools and procedures.

5.2.Recovery Potential and Impact on Digital Forensics

Deleted ext4 files may be recoverable depending on their size, how fragmented they are, how long it has been since they were deleted, and when they were removed. Due to a lack of in-built safe deletion procedures, it is commonly possible to retrieve deleted files from the ext4 file system [4].

File carving and disc imaging are two methods used by forensic investigators to retrieve erased data. In order to recover erased data, these techniques examine the disk's raw data in search of telltale file signatures or unique file structures. The data integrity of the lost file, the degree of fragmentation, and the amount of disc activity after the deletion all have a role in whether or not the file may be recovered.

There is a double repercussion for digital forensics. On the one hand, forensic analysts may be able to retrieve erased data, giving them access to potentially critical information that was either accidentally or purposefully obfuscated. This may be useful for retracing steps, gaining insight into user behaviours, and discovering previously unknown data.

However, forensic investigators face obstacles while attempting to retrieve permanently lost material. It may be difficult to determine the original filenames, creation dates, or ownership of recovered files if their information has been corrupted. It may take more time and effort to rebuild fragmented files. In addition, the amount of data that has to be analyzed might grow if deleted files are recoverable, which could extend the inquiry time.

If digital evidence is to be preserved and analyzed correctly, forensic analysts need to be aware of the consequences of file deletion and take the necessary precautions. Using the right forensic methods, keeping detailed records of the recovery procedure, and checking the data's validity and integrity are all essential steps.

6. Challenges in ext4 Forensics

When conducting digital investigations, forensic analysts have a number of difficulties when dealing with the ext4 file system. File fragmentation, missing or incorrect metadata, concealed information, and recovering lost data and artefacts are all problems that need to be addressed [5].

6.1.File Fragmentation and Data Reconstruction

When a file's data blocks are not saved in sequential order on the disc, this is known as file fragmentation. File fragmentation in ext4 may slow down data access and make it more difficult to rebuild files for forensics.

6.2.Incomplete Metadata and Hidden Data

It might be difficult to correctly determine file properties like filenames, creation dates, and ownership information when metadata is missing or incomplete. In order to find concealed information, forensic analysts must use a variety of investigation approaches, including reading the contents of files, mining metadata from unrelated sources, and slicing files.

6.3.Recovery of Deleted Files and Artifacts

Deleted files and artefacts may be recovered using the ext4 file system since the data is kept until it is rewritten. The possibility of recovering lost files is diminished by factors including file fragmentation, data overwriting, and insufficient metadata.

7. Conclusion

7.1.Summary of Findings

The purpose of this research was to analyze the ext4 file system by examining its foundations, including its data structures and the method of locating files. The results of file deletion were investigated, along with the difficulties posed by file fragmentation, insufficient metadata, concealed data, and the recovery of previously erased files and artefacts.

7.2.Importance of ext4 File System Analysis in Digital Forensics

Digital forensics investigations rely heavily on Ext4 file system analysis. By learning how ext4 works, forensic investigators may recover lost data, piece together what happened, and find out what was concealed. It's a great tool for discovering who's been doing what on the computer, when, and how.

By resolving the issues with ext4, forensic analysts can reliably extract digital evidence while protecting the integrity of the data and guaranteeing that it will be admissible in court. By analyzing Ext4 files, investigators may conduct more thorough and trustworthy forensic analyses.

7.3.Future Developments in ext4 Forensics

Future progress in ext4 forensics is anticipated as the file system continues to mature. Forensic investigations might be made more effective with the use of refined methods for analyzing file fragmentation and reconstructing lost data. Technological progress in metadata extraction and analysis of concealed data may eventually help forensic analysts unearth vital information. It's possible that specialized tools and procedures may be created to overcome the obstacles of retrieving lost data and artefacts.

Forensic analysts will be more equipped to deal with the ever-changing world of digital forensics and analyze ext4 file systems efficiently with further study and developments in ext4 forensics [6].

In conclusion, digital forensics investigations would be incomplete without ext4 file system analysis. Forensic analysts may still extract useful evidence, recreate events, and find buried information with a thorough grasp of ext4 and the deployment of relevant methods and tools. Forensic analysts may improve their investigation skills and aid in the successful settlement of computer abuse cases by adopting ext4 file system analysis.

References

- [1] S. L. a. H. H. J. Oh, " Forensic Recovery of File System Metadata for Digital Forensic Investigation," Vols. vol. 10, pp. 111591-111606, 2022, doi: 10.1109/ACCESS.2022.3213030..
- [2] T. Bagci, "What is Ext (Ext2, Ext3, Ext4) File System?," *SYSNETTECH Solutions*, no. <https://www.sysnettechsolutions.com/en/what-is-ext/>, January 3, 2021.
- [3] J. Corbet, "Securely deleting files from ext4 filesystems," *lwn.net*, October 11, 2011.
- [4] W. C. H. & S. T. Jo, "Digital forensic science approach by file recovery research. J Supercomput," Vols. 3704–3725 (2018). <https://doi.org/10.1007/s11227-016-1909-2>.
- [5] I. Owen, "Fairbanks, PhD, Kevin D. & Lee, Christopher Forensic implications of Ext4. ACM International Conference Proceeding Series.," vol. 22. 10.1145/1852666.1852691., 2010.
- [6] K. D. Fairbanks, "An analysis of Ext4 for digital forensics,Digital Investigation.," Vols. Volume 9, Supplement,2012,Pages S118-S130,ISSN 1742-2876, <https://doi.org/10.1016/j.diin.2012.05.010..>